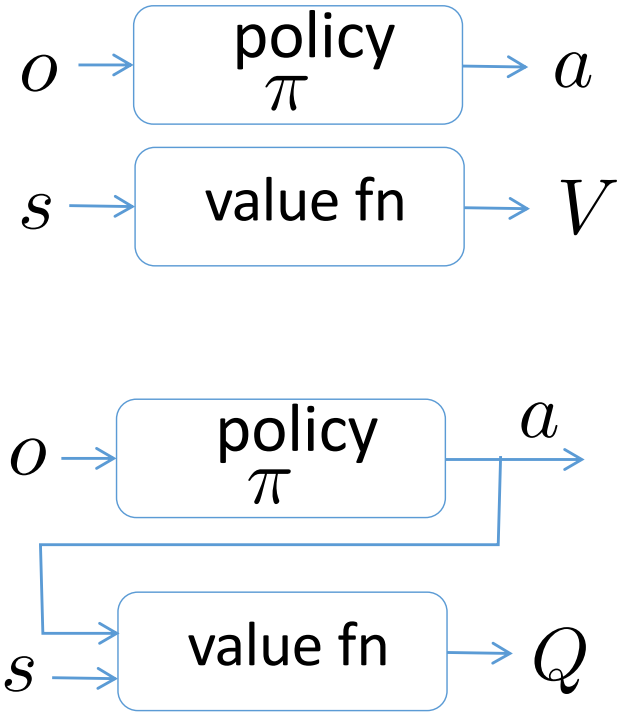
Common RL Tricks

- Asymmetric Actor Critic
- Privileged Learning
- DAgger
- Hindsight Experience Replay
- Solving POMDPs: Policies with Memory
- Dynamics Randomization

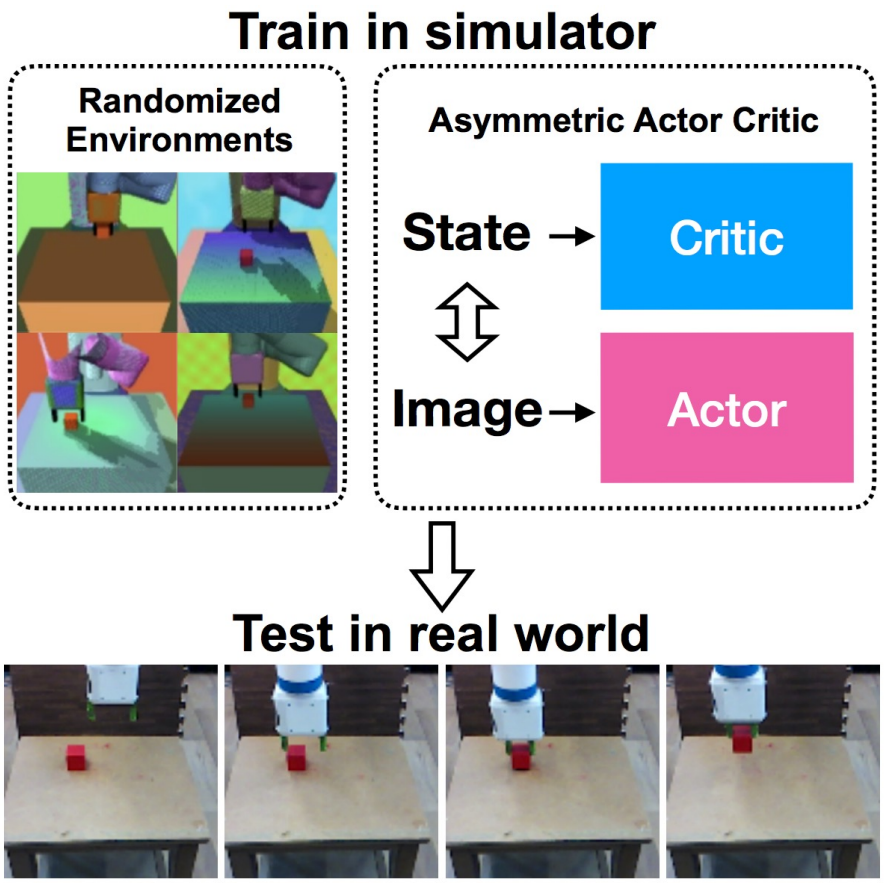
Asymmetric Actor Critic

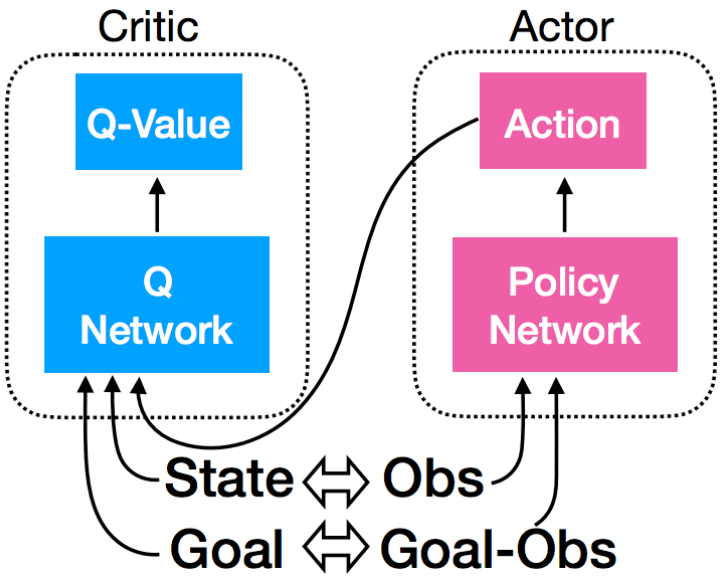
- make the policy easier to learn by giving the critic access to privileged information
- E.g.: policy needs to make decision from image observations, O, while the critic can access the state, s
- at run time, we don't need the critic!



Asymmetric Actor Critic for Image-Based Robot Learning [2017]

Lerrel Pinto[1,2] Marcin Andrychowicz[1] Peter Welinder[1] Wojciech Zaremba[1,†] Pieter Abbeel[1,†]





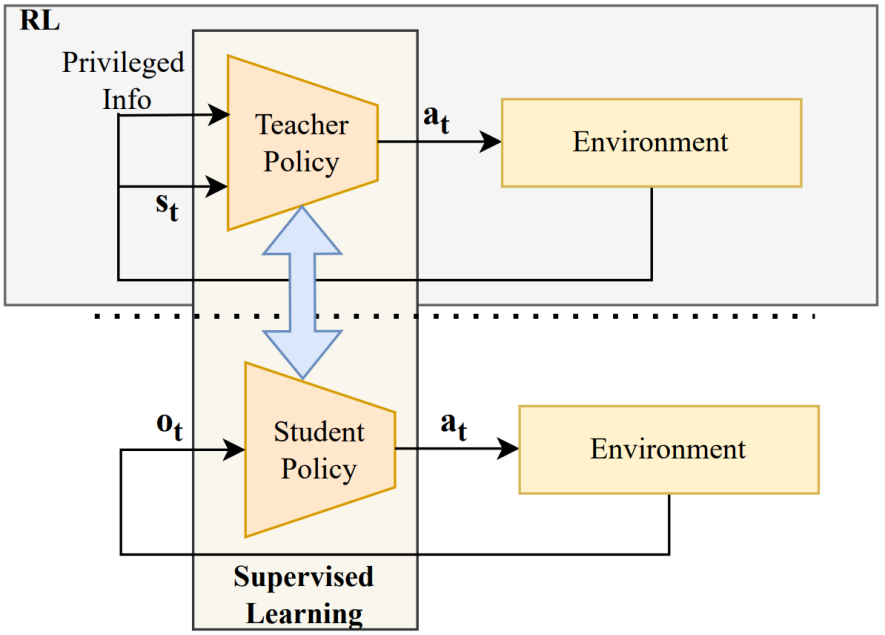
Can be used with any actor-critic algorithm; paper uses DDPG

Privileged Learning

("Learning by Cheating" [2020]; also heavily used in RL for quadrupeds)

teacher policy has access to the quadruped's state, including exact position + orientation, and perhaps a dense heightmap of surrounding terrain

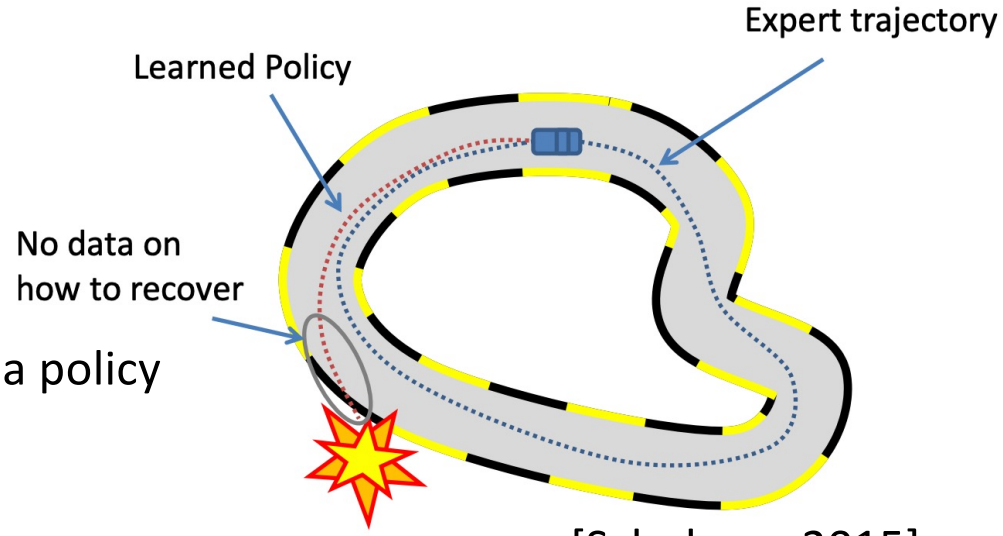
student has access to proprioceptive info, e.g., only joint positions & joint velocity, and perhaps some limited terrain info



- train teacher
- behavior cloning: run teacher policy & collect (s,a,o) data; student learns a = f(o)
- when possible: follow by DAgger

DAgger

"A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", Ross, Gordon & Bagnell (2010)



[Schulman 2015]

Useful anytime when transfering ("distilling") a policy

Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ **to** N **do**
  Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$.
  Sample T-step trajectories using π_i.
  Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.
  Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{D}_i$.
  Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$ (or use online learner to get $\hat{\pi}_{i+1}$ given new data $\mathcal{D}_i$).
end for
Return best $\hat{\pi}_i$ on validation.

typically: init STUDENT using behavior cloning:
- collect trajectories using EXPERT, supervised learning of STUDENT

collect trajectories using STUDENT policy

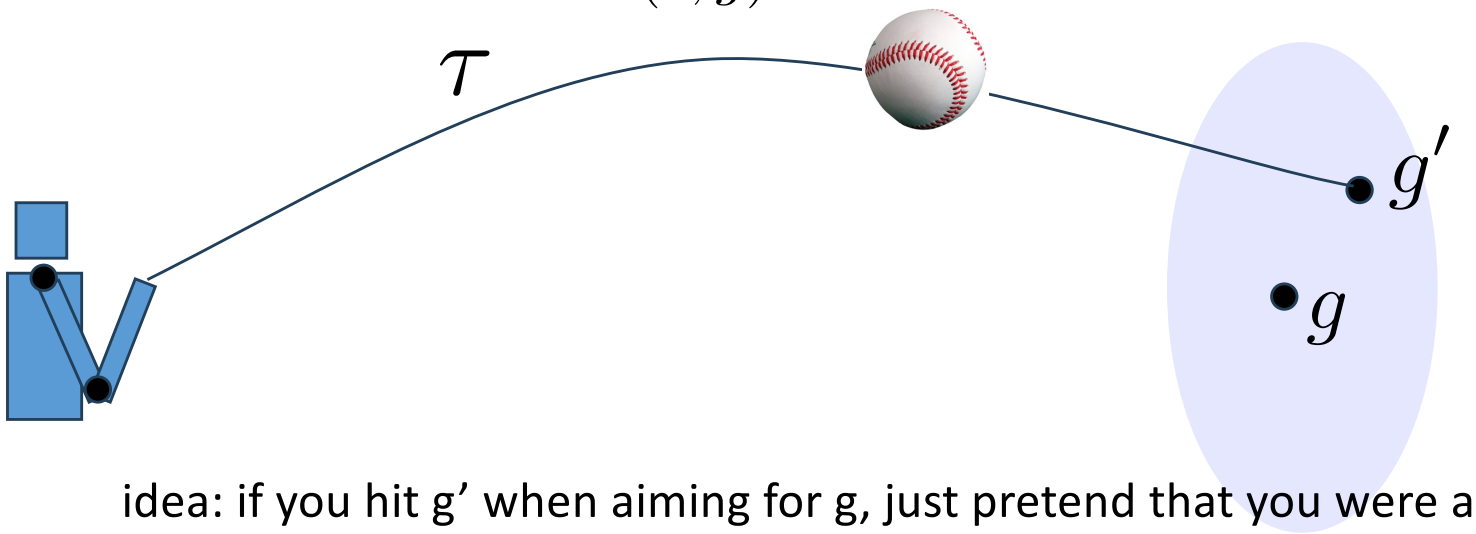
for the visited states, record the EXPERT actions

update the STUDENT policy (more behavior cloning)

Hindsight Experience Replay

- Useful for goal-conditioned policies: reward based on reaching goal

$a = \pi(s, g)$



idea: if you hit g' when aiming for g, just pretend that you were aiming for g'

$$a = \pi(s, g) \rightarrow a = \pi(s, g')$$

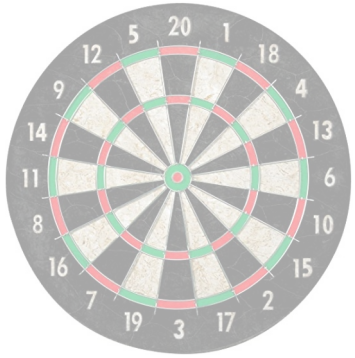
Just relabel all tuples:

$$(s_t, g', a_t, r'_t, s_{t+1}, g')$$

Hindsight Experience Replay

Marcin Andrychowicz*, **Filip Wolski**, **Alex Ray**, **Jonas Schneider**, **Rachel Fong**, **Peter Welinder**, **Bob McGrew**, **Josh Tobin**, **Pieter Abbeel**†, **Wojciech Zaremba**†
OpenAI

NeurIPS 2017



One ability humans have, unlike the current generation of model-free RL algorithms, is to learn almost as much from achieving an undesired outcome as from the desired one. Imagine that you are learning how to play hockey and are trying to shoot a puck into a net. You hit the puck but it misses the net on the right side. The conclusion drawn by a standard RL algorithm in such a situation would would be learned. It is however possible to draw another conclusion, namely that this sequence of actions would be successful if the net had been placed further to the right.

https://papers.nips.cc/paper/7090-hindsight-experience-replay.pdf

Algorithm 1 Hindsight Experience Replay (HER)

Given:

- an off-policy RL algorithm $\mathbb{A}$, ▷ e.g. DQN, DDPG, NAF, SDQN
- a strategy $\mathbb{S}$ for sampling goals for replay, ▷ e.g. $\mathbb{S}(s_0, \ldots, s_T) = m(s_T)$
- a reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$. ▷ e.g. $r(s, a, g) = -[f_g(s) = 0]$

Initialize $\mathbb{A}$ ▷ e.g. initialize neural networks

Initialize replay buffer R

for episode = 1, M **do**

  Sample a goal g and an initial state s_0.

  for $t = 0, T-1$ **do**

    Sample an action a_t using the behavioral policy from $\mathbb{A}$:

      $a_t \leftarrow \pi_b(s_t||g)$ ▷ $||$ denotes concatenation

    Execute the action a_t and observe a new state s_{t+1}

  end for

  for $t = 0, T-1$ **do**

    $r_t := r(s_t, a_t, g)$

    Store the transition $(s_t||g,\ a_t,\ r_t,\ s_{t+1}||g)$ in R ▷ standard experience replay

    Sample a set of additional goals for replay $G := \mathbb{S}(\textbf{current episode})$

    for $g' \in G$ **do**

      $r' := r(s_t, a_t, g')$

      Store the transition $(s_t||g',\ a_t,\ r',\ s_{t+1}||g')$ in R ▷ HER

    end for

  end for

  for $t = 1, N$ **do**

    Sample a minibatch B from the replay buffer R

    Perform one step of optimization using $\mathbb{A}$ and minibatch B

  end for

end for

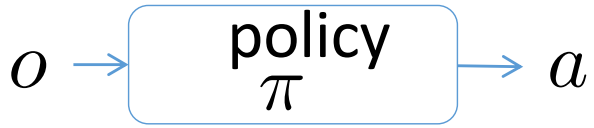
collect an epsiode

store in replay buffer:

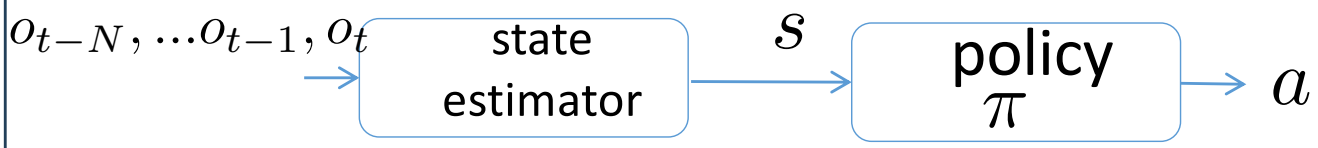
- original experiences
- hindsight experiences

off-policy training

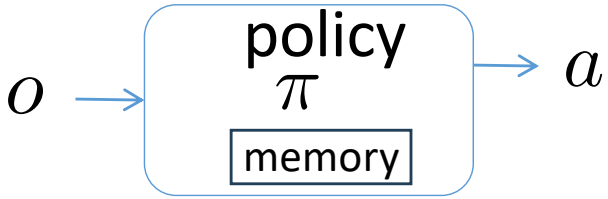
Solving POMDPs: Policies with Memory



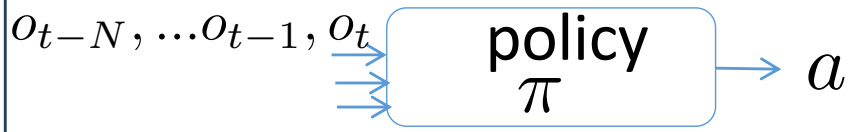
incomplete view of the world,
limited performance



use a state estimator



memory: RNN / LSTM / GRU
- recurrent neural network
- long-short-term memory
- gated recurrent unit



directly condition on observation history

Dynamics Randomization

- Sim-to-Real: trained policy → real robot ??
- idea: add random perturbations to key parameters (mass, friction, ...) for each episode.
- Policy will train to be robust to these pertubations
- Reality will hopefully look just like a perturbation

- Issue: may degrade performance

Sim-to-Real Transfer of Robotic Control with Dynamics Randomization

Xue Bin Peng[1,2], Marcin Andrychowicz[1], Wojciech Zaremba[1], and Pieter Abbeel[1,2] [2018]

C. Dynamics Randomization

During training, rollouts are organized into episodes of a fixed length. At the start of each episode, a random set of dynamics parameters μ are sampled according to ρ_μ and held fixed for the duration of the episode. The parameters which we randomize include:

- Mass of each link in the robot's body
- Damping of each joint
- Mass, friction, and damping of the puck
- Height of the table
- Gains for the position controller
- Timestep between actions
- Observation noise

which results in a total of 95 randomized parameters. The